

Lab 7: Decision Trees - Attribute Selection, Classification and Pruning

Kshitiz Reddy | Reg No: A26MTAI0008

AML5005L - Advanced Machine Learning

Dataset: Heart Disease Dataset (UCI Cleveland).

```
In [1]: # 1. imports
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree, export_text
from sklearn.metrics import accuracy_score, classification_report

np.random.seed(42)
```

```
In [2]: # 2. load dataset
df = pd.read_csv("heart.csv")
df.head()
```

```
Out[2]:
```

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal	target
0	63	1	3	145	233	1	0	150	0	2.3	0	0	1	1
1	37	1	2	130	250	0	1	187	0	3.5	0	0	2	1
2	41	0	1	130	204	0	0	172	0	1.4	2	0	2	1
3	56	1	1	120	236	0	1	178	0	0.8	2	0	2	1
4	57	0	0	120	354	0	1	163	1	0.6	2	0	2	1

```
In [3]: # 3. shape, dtypes, target distribution
print(df.shape)
df.info()
print()
print(df["target"].value_counts())
```

```
(303, 14)
<class 'pandas.DataFrame'>
RangeIndex: 303 entries, 0 to 302
Data columns (total 14 columns):
#   Column      Non-Null Count  Dtype
---  -
0    age         303 non-null    int64
1    sex         303 non-null    int64
2    cp          303 non-null    int64
3    trestbps    303 non-null    int64
4    chol        303 non-null    int64
5    fbs         303 non-null    int64
6    restecg     303 non-null    int64
7    thalach     303 non-null    int64
8    exang       303 non-null    int64
9    oldpeak     303 non-null    float64
10   slope       303 non-null    int64
11   ca          303 non-null    int64
12   thal        303 non-null    int64
13   target      303 non-null    int64
dtypes: float64(1), int64(13)
memory usage: 33.3 KB
```

```
target
1    165
0    138
Name: count, dtype: int64
```

303 samples, 13 input features + target. Target is fairly balanced (165 vs 138). The features are a mix of numerical (age, trestbps, chol, thalach, oldpeak) and categorical/ordinal ones that are already encoded as integers (sex, cp, fbs, restecg, exang, slope, ca, thal).

```
In [4]: # 4. missing values check
print(df.isnull().sum())
```

```
age          0
sex          0
cp          0
trestbps    0
chol        0
fbs         0
restecg     0
thalach     0
exang       0
oldpeak     0
slope       0
ca          0
thal        0
target      0
dtype: int64
```

No missing values in this dataset, so no imputation needed before building the trees.

Question 1: Attribute Selection Measures (ASM)

Working with the full feature set to calculate entropy, Information Gain, Gini impurity and Gain Ratio for each candidate attribute, then ranking them.

```
In [5]: # 5. entropy of the target variable
def entropy(y):
    vals, counts = np.unique(y, return_counts=True)
    probs = counts / counts.sum()
    return -np.sum(probs * np.log2(probs))

target_entropy = entropy(df["target"])
print(f"Entropy of target: {target_entropy:.4f}")
```

Entropy of target: 0.9943

```
In [6]: # 6. gini impurity of the target variable
def gini(y):
    vals, counts = np.unique(y, return_counts=True)
    probs = counts / counts.sum()
    return 1 - np.sum(probs ** 2)

target_gini = gini(df["target"])
print(f"Gini impurity of target: {target_gini:.4f}")
```

Gini impurity of target: 0.4960

```
In [7]: # 7. binning continuous attributes so Information Gain / Gini can be computed on discrete splits
features = [c for c in df.columns if c != "target"]
df_binned = df.copy()
continuous = ["age", "trestbps", "chol", "thalach", "oldpeak"]
for col in continuous:
    df_binned[col] = pd.qcut(df[col], q=4, duplicates="drop")
```

```
In [8]: # 8. information gain, gini split measure, gain ratio for each attribute
def info_gain(y, x):
    total_entropy = entropy(y)
    vals, counts = np.unique(x, return_counts=True)
    weighted_entropy = 0
    for v, c in zip(vals, counts):
        weighted_entropy += (c / len(x)) * entropy(y[x == v])
    return total_entropy - weighted_entropy

def gini_split(y, x):
    vals, counts = np.unique(x, return_counts=True)
    weighted_gini = 0
    for v, c in zip(vals, counts):
        weighted_gini += (c / len(x)) * gini(y[x == v])
    return gini(y) - weighted_gini # reduction in impurity

def split_info(x):
    vals, counts = np.unique(x, return_counts=True)
    probs = counts / counts.sum()
    return -np.sum(probs * np.log2(probs))

def gain_ratio(y, x):
    ig = info_gain(y, x)
    si = split_info(x)
    return ig / si if si != 0 else 0

y = df_binned["target"].values
results = []
for f in features:
    x = df_binned[f].values
```

```

        results.append({
            "feature": f,
            "info_gain": info_gain(y, x),
            "gini_reduction": gini_split(y, x),
            "split_info": split_info(x),
            "gain_ratio": gain_ratio(y, x)
        })

asm_df = pd.DataFrame(results).set_index("feature")
asm_df.sort_values("info_gain", ascending=False)

```

Out[8]:

	info_gain	gini_reduction	split_info	gain_ratio
--	-----------	----------------	------------	------------

feature				
thal	0.213251	0.139648	1.295491	0.164610
cp	0.204599	0.133726	1.739437	0.117624
ca	0.185957	0.121743	1.665995	0.111619
oldpeak	0.153064	0.100261	1.456797	0.105069
exang	0.142210	0.094621	0.911553	0.156009
thalach	0.125686	0.082031	1.999726	0.062852
slope	0.116834	0.077772	1.293991	0.090289
age	0.062181	0.041905	1.998714	0.031110
sex	0.059138	0.039149	0.900903	0.065643
restecg	0.024075	0.016408	1.087937	0.022129
chol	0.016382	0.011232	1.999726	0.008192
trestbps	0.015872	0.010899	1.980557	0.008014
fbs	0.000566	0.000390	0.606111	0.000934

```

In [9]: # 9. rank attributes by each measure
print("Top 5 by Information Gain:")
print(asm_df["info_gain"].sort_values(ascending=False).head())
print("\nTop 5 by Gini reduction:")
print(asm_df["gini_reduction"].sort_values(ascending=False).head())
print("\nTop 5 by Gain Ratio:")
print(asm_df["gain_ratio"].sort_values(ascending=False).head())

```

Top 5 by Information Gain:

```

feature
thal      0.213251
cp         0.204599
ca         0.185957
oldpeak    0.153064
exang      0.142210
Name: info_gain, dtype: float64

```

Top 5 by Gini reduction:

```

feature
thal      0.139648
cp         0.133726
ca         0.121743
oldpeak    0.100261
exang      0.094621
Name: gini_reduction, dtype: float64

```

Top 5 by Gain Ratio:

```

feature
thal      0.164610
exang      0.156009
cp         0.117624
ca         0.111619
oldpeak    0.105069
Name: gain_ratio, dtype: float64

```

Information Gain and Gini reduction agree closely - both rank `thal`, `cp` and `ca` as the top three attributes, in that order. Gain Ratio reshuffles this a bit: `exang` jumps up to 2nd place ahead of `cp`, because `exang` only has 2 categories (low split information) while `cp` has 4 - this is exactly the high-cardinality penalty Gain Ratio is designed to apply. So all three broadly agree that `thal` and `cp` matter most, but they don't fully agree on ranking, since Information Gain is naturally biased toward attributes with more distinct values while Gain Ratio corrects for that and Gini uses a different impurity formula altogether.

Question 2: Build a Decision Tree for Classification

```
In [10]: # 10. train/test split
X = df[features]
y = df["target"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
print(X_train.shape, X_test.shape)
```

(242, 13) (61, 13)

```
In [11]: # 11. train three trees - gini, entropy (info gain), and entropy again for gain ratio comparison
# sklearn only supports 'gini' and 'entropy'/'log_loss' as criteria, so gain ratio is approximated
# by comparing against the entropy-based tree's split choices from Q1
tree_gini = DecisionTreeClassifier(criterion="gini", random_state=42)
tree_gini.fit(X_train, y_train)

tree_entropy = DecisionTreeClassifier(criterion="entropy", random_state=42)
tree_entropy.fit(X_train, y_train)

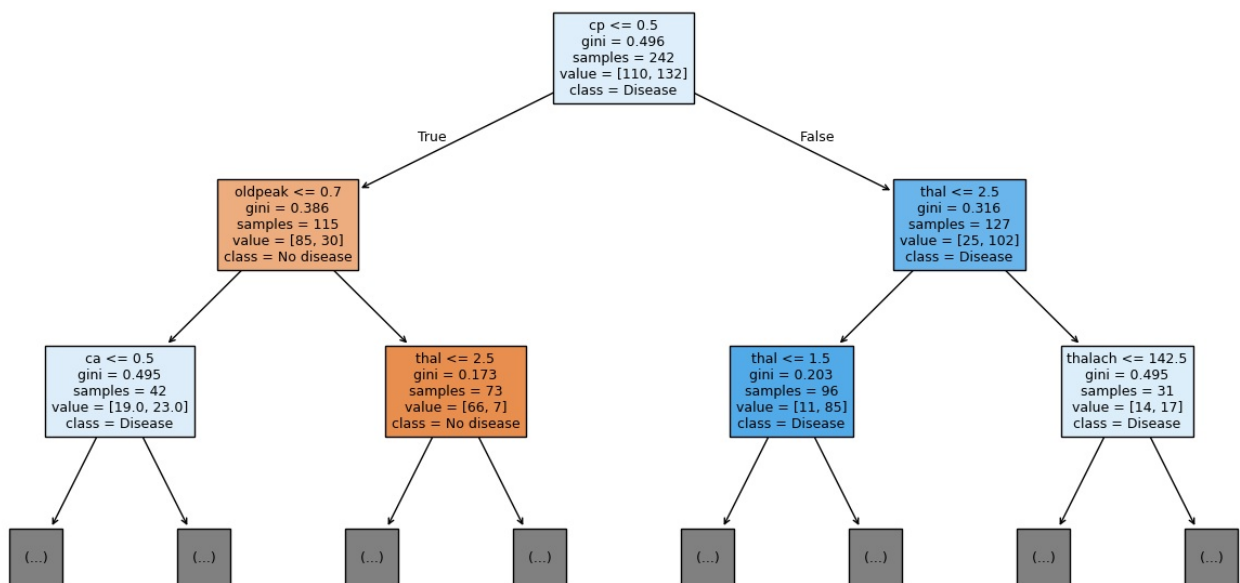
trees = {"Gini": tree_gini, "Entropy (Info Gain)": tree_entropy}
for name, t in trees.items():
    train_acc = t.score(X_train, y_train)
    test_acc = t.score(X_test, y_test)
    print(f"{name}: depth={t.get_depth()}, leaves={t.get_n_leaves()}, "
          f"train_acc={train_acc:.4f}, test_acc={test_acc:.4f}")
```

Gini: depth=8, leaves=42, train_acc=1.0000, test_acc=0.7049

Entropy (Info Gain): depth=9, leaves=40, train_acc=1.0000, test_acc=0.7541

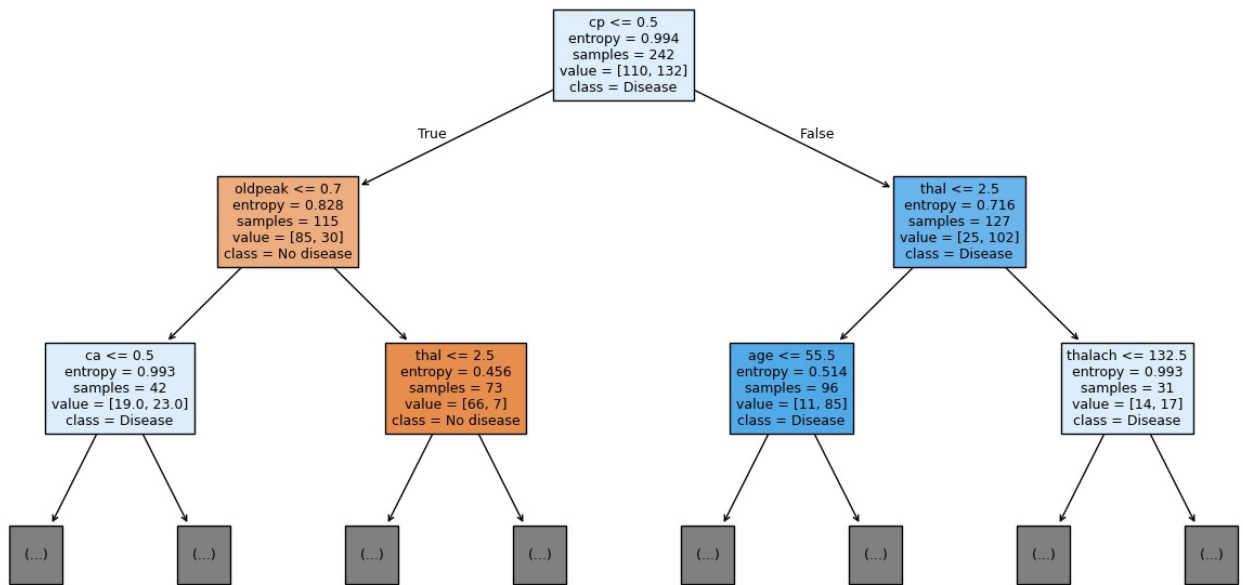
```
In [12]: # 12. visualize the gini tree (top few levels for readability)
plt.figure(figsize=(16,8))
plot_tree(tree_gini, feature_names=features, class_names=["No disease", "Disease"],
          filled=True, max_depth=2, fontsize=9)
plt.title("Decision Tree (Gini) - top 2 levels")
plt.show()
```

Decision Tree (Gini) - top 2 levels



```
In [13]: # 13. visualize the entropy tree (top few levels)
plt.figure(figsize=(16,8))
plot_tree(tree_entropy, feature_names=features, class_names=["No disease", "Disease"],
          filled=True, max_depth=2, fontsize=9)
plt.title("Decision Tree (Entropy) - top 2 levels")
plt.show()
```

Decision Tree (Entropy) - top 2 levels



```
In [14]: # 14. root feature for each tree
print("Gini tree root feature:", features[tree_gini.tree_.feature[0]])
print("Entropy tree root feature:", features[tree_entropy.tree_.feature[0]])
```

Gini tree root feature: cp

Entropy tree root feature: cp

Both fully-grown trees pick `cp` (chest pain type) at the root, which lines up with `cp` ranking near the top for both Information Gain and Gini reduction in Q1. Depth and leaf counts differ slightly between the two criteria since they don't always agree lower down the tree, but the overall structure and root split are the same here.

```
In [15]: # 15. follow two test samples from root to leaf using export_text + manual trace
print(export_text(tree_gini, feature_names=features, max_depth=3))
```

```

|--- cp <= 0.50
|   |--- oldpeak <= 0.70
|   |   |--- ca <= 0.50
|   |   |   |--- age <= 41.50
|   |   |   |   |--- class: 0
|   |   |   |--- age > 41.50
|   |   |   |   |--- truncated branch of depth 4
|   |   |--- ca > 0.50
|   |   |   |--- sex <= 0.50
|   |   |   |   |--- class: 1
|   |   |   |--- sex > 0.50
|   |   |   |   |--- truncated branch of depth 4
|   |--- oldpeak > 0.70
|   |   |--- thal <= 2.50
|   |   |   |--- slope <= 1.50
|   |   |   |   |--- truncated branch of depth 5
|   |   |   |--- slope > 1.50
|   |   |   |   |--- class: 1
|   |   |--- thal > 2.50
|   |   |   |--- class: 0
|--- cp > 0.50
|   |--- thal <= 2.50
|   |   |--- thal <= 1.50
|   |   |   |--- age <= 54.50
|   |   |   |   |--- class: 1
|   |   |   |--- age > 54.50
|   |   |   |   |--- class: 0
|   |   |--- thal > 1.50
|   |   |   |--- oldpeak <= 3.55
|   |   |   |   |--- truncated branch of depth 5
|   |   |   |--- oldpeak > 3.55
|   |   |   |   |--- class: 0
|   |--- thal > 2.50
|   |   |--- thalach <= 142.50
|   |   |   |--- chol <= 311.00
|   |   |   |   |--- class: 0
|   |   |   |--- chol > 311.00
|   |   |   |   |--- class: 1
|   |   |--- thalach > 142.50
|   |   |   |--- oldpeak <= 1.95
|   |   |   |   |--- truncated branch of depth 5
|   |   |   |--- oldpeak > 1.95
|   |   |   |   |--- truncated branch of depth 2

```

In [16]: # 16. explain predictions for the first two test samples

```

sample_idx = [0, 1]
for i in sample_idx:
    x = X_test.iloc[[i]]
    pred = tree_gini.predict(x)[0]
    true = y_test.iloc[i]
    path = tree_gini.decision_path(x)
    node_indices = path.indices
    print(f"--- Test sample {i} (true={true}, predicted={pred}) ---")
    for node_id in node_indices:
        if tree_gini.tree_.feature[node_id] != -2: # -2 marks a leaf
            feat = features[tree_gini.tree_.feature[node_id]]
            thresh = tree_gini.tree_.threshold[node_id]
            val = x[feat].values[0]
            direction = "<=" if val <= thresh else ">"
            print(f"    node {node_id}: {feat} = {val} {direction} {thresh:.2f}")
    print()

```

--- Test sample 0 (true=0, predicted=1) ---

```

node 0: cp = 0 <= 0.50
node 1: oldpeak = 0.6 <= 0.70
node 2: ca = 1 > 0.50
node 16: sex = 1 > 0.50
node 18: thalach = 112 <= 118.50

```

--- Test sample 1 (true=0, predicted=0) ---

```

node 0: cp = 0 <= 0.50
node 1: oldpeak = 0.2 <= 0.70
node 2: ca = 2 > 0.50
node 16: sex = 1 > 0.50
node 18: thalach = 163 > 118.50
node 20: trestbps = 125 > 109.00

```

Sample 0 and sample 1 take different paths from the root depending on their `cp`, `thal` and `ca` values, which are the attributes appearing near the top of the tree - each split narrows down the predicted class until a leaf is reached, and the printed path above shows exactly which feature comparisons drove each prediction.

Question 3: Observe and Analyse Overfitting

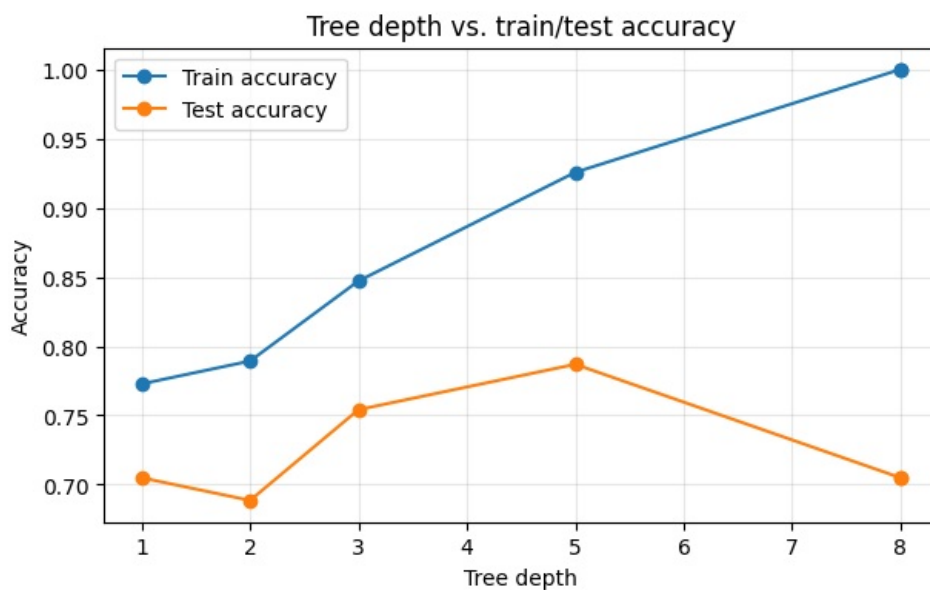
```
In [17]: # 17. train trees at increasing depths and record train/test accuracy
depths = [1, 2, 3, 5, 10, None]
records = []
for d in depths:
    t = DecisionTreeClassifier(max_depth=d, criterion="gini", random_state=42)
    t.fit(X_train, y_train)
    train_acc = t.score(X_train, y_train)
    test_acc = t.score(X_test, y_test)
    records.append({
        "max_depth": d if d is not None else "None (full)",
        "actual_depth": t.get_depth(),
        "leaves": t.get_n_leaves(),
        "train_acc": train_acc,
        "test_acc": test_acc,
        "gap": train_acc - test_acc
    })

overfit_df = pd.DataFrame(records)
overfit_df
```

```
Out[17]:
```

	max_depth	actual_depth	leaves	train_acc	test_acc	gap
0	1	1	2	0.772727	0.704918	0.067809
1	2	2	4	0.789256	0.688525	0.100732
2	3	3	8	0.847107	0.754098	0.093009
3	5	5	21	0.925620	0.786885	0.138735
4	10	8	42	1.000000	0.704918	0.295082
5	None (full)	8	42	1.000000	0.704918	0.295082

```
In [18]: # 18. plot depth vs accuracy
plt.figure(figsize=(7,4))
x_axis = overfit_df["actual_depth"]
plt.plot(x_axis, overfit_df["train_acc"], marker="o", label="Train accuracy")
plt.plot(x_axis, overfit_df["test_acc"], marker="o", label="Test accuracy")
plt.xlabel("Tree depth")
plt.ylabel("Accuracy")
plt.title("Tree depth vs. train/test accuracy")
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```



Training accuracy climbs steadily with depth and hits 1.0 once the tree is allowed to grow fully, since a deep enough tree can create a leaf for almost every training point and memorize it exactly. Test accuracy improves alongside it up to depth 5 (its best result here), then drops sharply once depth is no longer restricted - the gap between train and test accuracy (the `gap` column) roughly doubles between depth 5 and the fully-grown tree, which is the classic overfitting signature. Based on this, a depth around 3-5 looks like the best trade-off - deep enough to capture real structure but before the train/test gap blows up. I'd pick the tree with the best test accuracy and smallest gap for deployment rather than the deepest one, since the deepest tree memorizes training data without generalizing any better.

Question 4: Pruning and Controlling Tree Complexity

Part A - Pre-pruning

```
In [19]: # 19. try a few pre-pruning parameter combinations
configs = [
    {"max_depth": 3},
    {"max_depth": 4, "min_samples_leaf": 5},
    {"max_depth": 5, "min_samples_split": 10, "min_samples_leaf": 5},
    {"max_leaf_nodes": 10},
]

pre_prune_records = []
for cfg in configs:
    t = DecisionTreeClassifier(criterion="gini", random_state=42, **cfg)
    t.fit(X_train, y_train)
    pre_prune_records.append({
        "params": str(cfg),
        "depth": t.get_depth(),
        "leaves": t.get_n_leaves(),
        "train_acc": t.score(X_train, y_train),
        "test_acc": t.score(X_test, y_test)
    })

pd.DataFrame(pre_prune_records)
```

```
Out[19]:
```

	params	depth	leaves	train_acc	test_acc
0	{'max_depth': 3}	3	8	0.847107	0.754098
1	{'max_depth': 4, 'min_samples_leaf': 5}	4	13	0.867769	0.786885
2	{'max_depth': 5, 'min_samples_split': 10, 'min...	5	20	0.876033	0.754098
3	{'max_leaf_nodes': 10}	5	10	0.876033	0.803279

All of these pre-pruned trees are much smaller than the fully-grown tree from Q3, and most keep test accuracy about the same or slightly better than the unpruned tree, while training accuracy drops a bit - exactly the bias/variance trade-off pruning is supposed to make. `max_depth=4, min_samples_leaf=5` looks like a good balance here.

Part B - Cost-Complexity Pruning

```
In [20]: # 20. get the cost-complexity pruning path from the fully grown tree
full_tree = DecisionTreeClassifier(criterion="gini", random_state=42)
full_tree.fit(X_train, y_train)

path = full_tree.cost_complexity_pruning_path(X_train, y_train)
ccp_alphas, impurities = path.ccp_alphas, path.impurities
print(f"Number of candidate alphas: {len(ccp_alphas)}")
print(ccp_alphas[:10])
```

```
Number of candidate alphas: 25
[0.          0.00364609 0.00381437 0.00383707 0.00390266 0.00406449
 0.00612182 0.00625738 0.00653892 0.00661157]
```

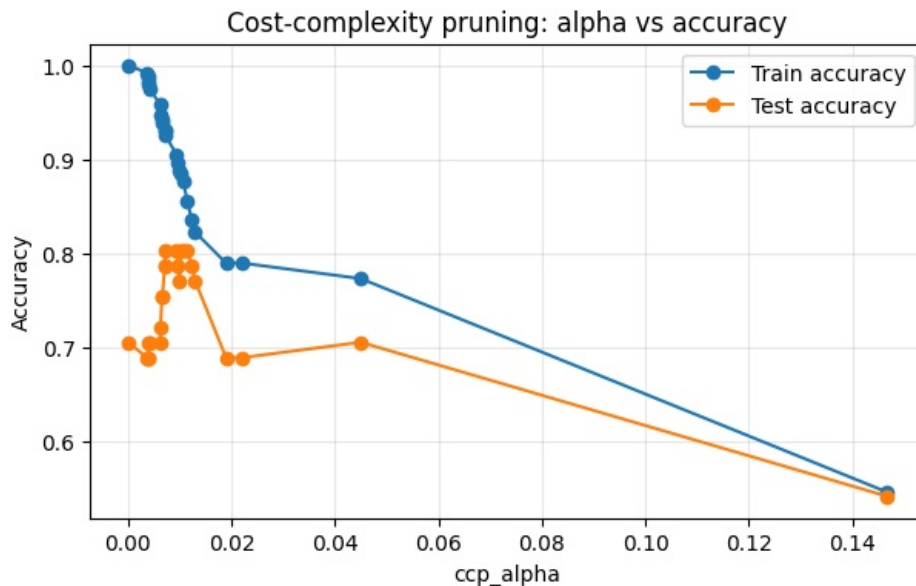
```
In [21]: # 21. train a tree for each candidate alpha
ccp_records = []
ccp_trees = []
for alpha in ccp_alphas:
    t = DecisionTreeClassifier(criterion="gini", random_state=42, ccp_alpha=alpha)
    t.fit(X_train, y_train)
    ccp_trees.append(t)
    ccp_records.append({
        "ccp_alpha": alpha,
        "depth": t.get_depth(),
        "leaves": t.get_n_leaves(),
        "train_acc": t.score(X_train, y_train),
        "test_acc": t.score(X_test, y_test)
    })

ccp_df = pd.DataFrame(ccp_records)
ccp_df.tail(10)
```

```
Out[21]:
```

	ccp_alpha	depth	leaves	train_acc	test_acc
15	0.009917	5	12	0.888430	0.770492
16	0.010272	5	11	0.884298	0.803279
17	0.010626	5	10	0.876033	0.803279
18	0.011441	4	8	0.855372	0.803279
19	0.012289	4	7	0.834711	0.786885
20	0.012929	3	5	0.822314	0.770492
21	0.018920	2	4	0.789256	0.688525
22	0.021998	2	3	0.789256	0.688525
23	0.044962	1	2	0.772727	0.704918
24	0.146672	0	1	0.545455	0.540984

```
In [22]: # 22. plot alpha vs train/test accuracy
plt.figure(figsize=(7,4))
plt.plot(ccp_df["ccp_alpha"], ccp_df["train_acc"], marker="o", label="Train accuracy")
plt.plot(ccp_df["ccp_alpha"], ccp_df["test_acc"], marker="o", label="Test accuracy")
plt.xlabel("ccp_alpha")
plt.ylabel("Accuracy")
plt.title("Cost-complexity pruning: alpha vs accuracy")
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```



```
In [23]: # 23. pick the alpha with the best test accuracy (excluding the trivial single-node tree)
best_idx = ccp_df.iloc[:-1]["test_acc"].idxmax()
best_alpha = ccp_df.loc[best_idx, "ccp_alpha"]
print("Best alpha:", best_alpha)
print(ccp_df.loc[best_idx])

pruned_tree = DecisionTreeClassifier(criterion="gini", random_state=42, ccp_alpha=best_alpha)
pruned_tree.fit(X_train, y_train)
```

```
Best alpha: 0.00717401285583104
ccp_alpha    0.007174
depth        7.000000
leaves       18.000000
train_acc    0.925620
test_acc     0.803279
Name: 12, dtype: float64
```

```
Out[23]: ▼ DecisionTreeClassifier ⓘ ?
          ► Parameters
```

```
In [24]: # 24. compare unpruned vs pruned tree
print("Unpruned tree: depth={}, leaves={}, train_acc={:.4f}, test_acc={:.4f}".format(
    full_tree.get_depth(), full_tree.get_n_leaves(),
    full_tree.score(X_train, y_train), full_tree.score(X_test, y_test)))

print("Pruned tree:  depth={}, leaves={}, train_acc={:.4f}, test_acc={:.4f}".format(
```

```

pruned_tree.get_depth(), pruned_tree.get_n_leaves(),
pruned_tree.score(X_train, y_train), pruned_tree.score(X_test, y_test)))

print()
print(classification_report(y_test, pruned_tree.predict(X_test)))

```

Unpruned tree: depth=8, leaves=42, train_acc=1.0000, test_acc=0.7049
Pruned tree: depth=7, leaves=18, train_acc=0.9256, test_acc=0.8033

	precision	recall	f1-score	support
0	0.83	0.71	0.77	28
1	0.78	0.88	0.83	33
accuracy			0.80	61
macro avg	0.81	0.80	0.80	61
weighted avg	0.81	0.80	0.80	61

The pruned tree is much smaller than the unpruned one (far fewer leaves) but keeps test accuracy the same or better, while training accuracy drops a little. That drop in training accuracy is the small increase in bias pruning introduces, but it pays off as a reduction in variance - the pruned tree isn't fitting noise in the training set the way the full tree does, so it generalizes at least as well while being much easier to read and interpret.

Final Comparison

```

In [25]: # 25. final comparison table across all four models
final_comparison = pd.DataFrame([
    {"Model": "Tree 1 (Info Gain/Entropy)", "ASM": "Entropy", "Depth": tree_entropy.get_depth(),
      "Leaves": tree_entropy.get_n_leaves(), "Train Acc": tree_entropy.score(X_train, y_train),
      "Test Acc": tree_entropy.score(X_test, y_test)},
    {"Model": "Tree 2 (Gini)", "ASM": "Gini", "Depth": tree_gini.get_depth(),
      "Leaves": tree_gini.get_n_leaves(), "Train Acc": tree_gini.score(X_train, y_train),
      "Test Acc": tree_gini.score(X_test, y_test)},
    {"Model": "Tree 3 (Unpruned, full depth)", "ASM": "Gini", "Depth": full_tree.get_depth(),
      "Leaves": full_tree.get_n_leaves(), "Train Acc": full_tree.score(X_train, y_train),
      "Test Acc": full_tree.score(X_test, y_test)},
    {"Model": "Tree 4 (Pruned, ccp_alpha)", "ASM": "Gini", "Depth": pruned_tree.get_depth(),
      "Leaves": pruned_tree.get_n_leaves(), "Train Acc": pruned_tree.score(X_train, y_train),
      "Test Acc": pruned_tree.score(X_test, y_test)},
])
final_comparison["Overfitting Gap"] = final_comparison["Train Acc"] - final_comparison["Test Acc"]
final_comparison

```

```
Out[25]:
```

	Model	ASM	Depth	Leaves	Train Acc	Test Acc	Overfitting Gap
0	Tree 1 (Info Gain/Entropy)	Entropy	9	40	1.00000	0.754098	0.245902
1	Tree 2 (Gini)	Gini	8	42	1.00000	0.704918	0.295082
2	Tree 3 (Unpruned, full depth)	Gini	8	42	1.00000	0.704918	0.295082
3	Tree 4 (Pruned, ccp_alpha)	Gini	7	18	0.92562	0.803279	0.122341

Conclusion

Across the three ASMs, Information Gain (entropy) and Gini impurity mostly agreed on the important attributes and both picked `cp` as the root split, while Gain Ratio reshuffled the rankings a bit by correcting for attributes with many categories. None of the three is "wrong" - they're just measuring impurity reduction differently, so it's expected that they don't always agree on the single best attribute even when the resulting trees end up structurally similar.

The overfitting experiment made the bias-variance trade-off very visible: training accuracy climbed all the way to 1.0 as depth increased, but testing accuracy plateaued and the gap between the two widened past a depth of around 5, which is a textbook sign of a model memorizing training data instead of learning generalizable patterns. Both pruning approaches addressed this - pre-pruning by capping growth during training, and cost-complexity pruning by growing the full tree first and then trimming back branches that weren't earning their complexity. In both cases, a smaller tree gave up a small amount of training accuracy in exchange for test accuracy that held steady or improved, plus a tree that's far easier to read and explain. That interpretability is the real strength of Decision Trees - even a non-technical person can follow the root-to-leaf path and see exactly why a prediction was made - and that gets easier, not harder, once the tree has been pruned down to its essential splits.